

Unsupervised Image Segmentation and Denoising

Outline

- 1 Introduction
- 2 Related Work
- 3 TokenCut Methodology
- 4 EAGLE Methodology
- 5 Experimental Setup
- 6 Results & Evaluation
- 7 Comparative Analysis
- 8 Conclusion & Future Work
- 9 References

The Problem: Unsupervised Segmentation

Definition

The task of partitioning an image into semantically meaningful regions (objects vs. background) **without** using any pixel-level annotations during training.

The Challenge:

- Without labels, models must rely on intrinsic image properties
- Color, texture, edges or self-supervised features
- Must discover objects autonomously

Why Unsupervised Segmentation?

Scalability

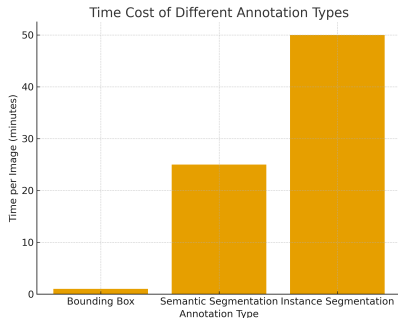
- Manual pixel-wise annotation is expensive
- Time-consuming for large datasets

Generalization

- Not restricted to predefined classes
- Can discover novel object categories

Discovery

- Useful for exploring unlabeled data
- Enables zero-shot segmentation



Binary Segmentation Focus

Distinguishing between:

- **"Things" (Foreground):** Countable objects (person, car, bird)
- **"Stuff" (Background):** Amorphous regions (sky, grass, road)

Project Objectives:

- 1 Implement **TokenCut**: Training-free, graph-based method
- 2 Implement **EAGLE**: Self-supervised contrastive learning
- 3 Compare on COCO-Stuff-27 dataset
- 4 Analyze accuracy vs. efficiency trade-offs

Traditional Methods

Graph-Based Segmentation

Normalized Cuts (Shi & Malik, 2000)

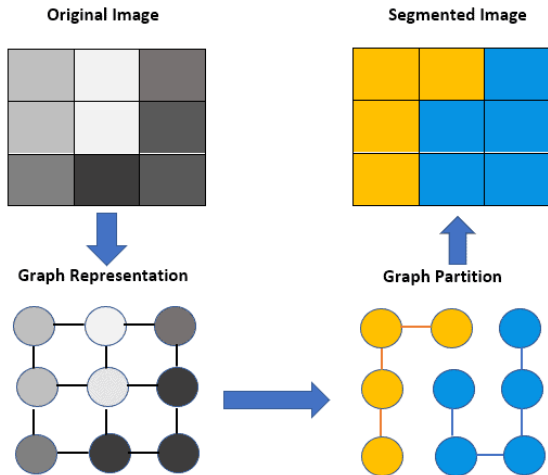
- Treats segmentation as graph partitioning
- Pixels = nodes, edges = similarity
- Minimize cut cost while maximizing internal association
- *Limitation:* Relies on low-level features (RGB, gradients)

Clustering Methods

K-Means / Mean Shift

- Clusters pixels in color space
- *Limitation:* No semantic understanding; over-segments textured objects

Graph Based Segmentation Example



Deep Learning Era

Supervised Learning

- FCN, U-Net, DeepLab achieve SOTA results
- Require massive annotated datasets

Self-Supervised Learning (SSL)

- **Contrastive Learning:** SimCLR, MoCo
- **Vision Transformers & DINO:**
 - DINO = Distillation with No Labels
 - Self-attention maps highlight salient objects
 - Both TokenCut and EAGLE build on DINO features



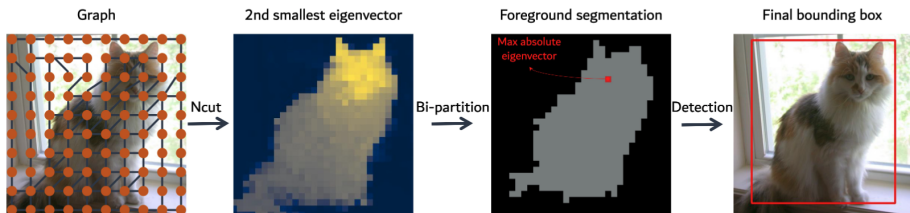
TokenCut: Overview

Core Concept

"Segmenting Objects without Training" - Leverages DINO features + Normalized Cuts

Workflow:

- 1 Extract features from pre-trained DINO model
- 2 Construct graph where nodes = image patches
- 3 Solve spectral clustering to find object



DINO ViT-Small

- Vision Transformer pre-trained with DINO
- **Input:** Image $I \in \mathbb{R}^{H \times W \times 3}$
- **Patches:** Split into 16×16 patches
- **Features:** $F \in \mathbb{R}^{N \times D}$
 - N = number of patches
 - D = feature dimension

Why DINO?

DINO features exhibit **semantic consistency**. Patches from the same object (e.g., dog head + tail) have high cosine similarity, even if RGB values differ.

Similarity Graph $G = (V, E)$

- **Nodes (V):** The N image patches
- **Edges (E):** Weighted by similarity

Affinity Matrix (A):

$$A_{ij} = \begin{cases} \cos(f_i, f_j) & \text{if } \cos(f_i, f_j) > \tau \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

- τ : Threshold to remove weak connections
- Captures **semantic relationship** between patch pairs

TokenCut: Normalized Cut Problem

Objective: Partition vertices V into disjoint sets A (Foreground) and B (Background)

$$Ncut(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)} \quad (2)$$

Where:

- $cut(A, B) = \sum_{i \in A, j \in B} w_{ij}$ (sum of edge weights crossing the cut)
- $assoc(A, V) = \sum_{i \in A} d_i$ (total connection from A to all nodes)

Intuition

Cut *weak* edges (low similarity) such that remaining partitions are *strongly* connected internally.

TokenCut: Solving the N-Cut

Spectral Relaxation (NP-hard $\rightarrow$ Eigenvalue problem):

$$D^{-\frac{1}{2}}(D - A)D^{-\frac{1}{2}}z = \lambda z \quad (3)$$

- D : Degree matrix (diagonal, $D_{ii} = \sum_j A_{ij}$)
- $L = D - A$: Laplacian matrix
- $\mathcal{L}_{sym} = D^{-\frac{1}{2}}LD^{-\frac{1}{2}}$: Normalized Laplacian

Solution

The **second smallest eigenvector** (Fiedler vector) of $\mathcal{L}_{sym}$ provides the optimal real-valued bipartition. Threshold at mean or 0 to get binary mask.

Algorithm 1 TokenCut

- 1: **Input:** Image I
 - 2: Extract features F using DINO ViT-S/16
 - 3: Compute affinity $A = FF^\top$ (cosine similarity)
 - 4: Compute degree matrix D and $\mathcal{L}_{sym}$
 - 5: Solve $\mathcal{L}_{sym}z = \lambda z$ for 2nd smallest eigenvector y
 - 6: Threshold y to obtain binary mask M
 - 7: **Optional:** Refine with bilateral solver or CRF
 - 8: **Output:** Segmentation mask M
-

TokenCut Evaluation Datasets

ECSSD Dataset

Extended Complex Scene Saliency

- **Size:** 1,000 images
- **Content:** Natural scenes with prominent objects
- **Annotations:** Pixel-wise binary masks

Characteristics:

- Clear figure-ground separation
- Centered salient objects
- Varying complexity levels

CUB-200-2011 Dataset

Caltech-UCSD Birds

- **Size:** 11,788 images
- **Classes:** 200 bird species
- **Annotations:** Segmentation masks + keypoints

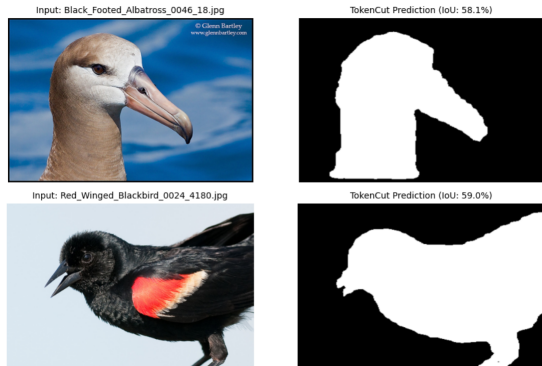
Challenges:

- Complex feather textures
- Thin structures (legs, beaks)
- Multi-colored plumage

ECSSD Dataset



CUB-200-2011 Dataset



Dataset	mIoU
ECSSD	42.82%
CUB-200	32.97%
COCO-Stuff-27	31.75%

Table: TokenCut Performance

Performance Analysis:

Strong Performance (ECSSD)

- High-contrast objects
- DINO features align well

Limitations

- **Color sensitivity:** Fragments multi-colored objects
- **Graph-cut bias:** Prefers single dominant object

Eigen Aggregation Learning

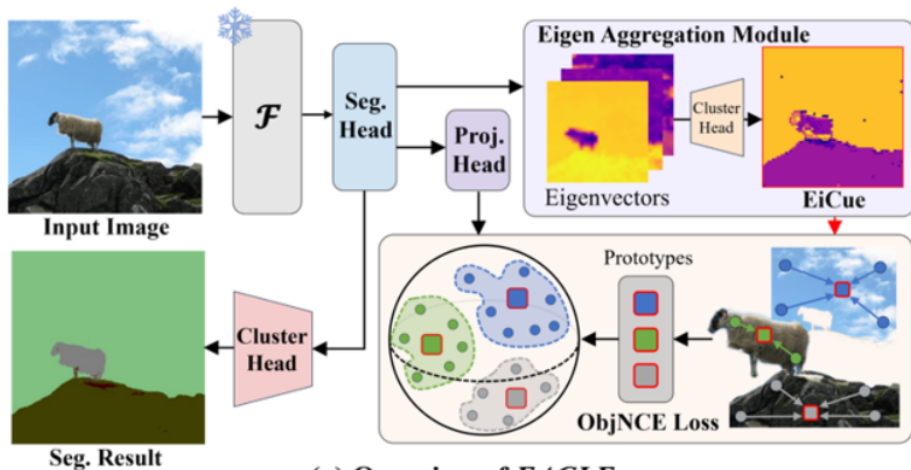
Addresses the "**Chicken-and-Egg**" problem:

- Good segmentation needs good features
- Good features need good segmentation

Solution: Iterative refinement

- 1 Use spectral priors (**EiCue**) to initialize masks
- 2 Use contrastive learning (**ObjNCELoss**) to refine features

EAGLE: Overview

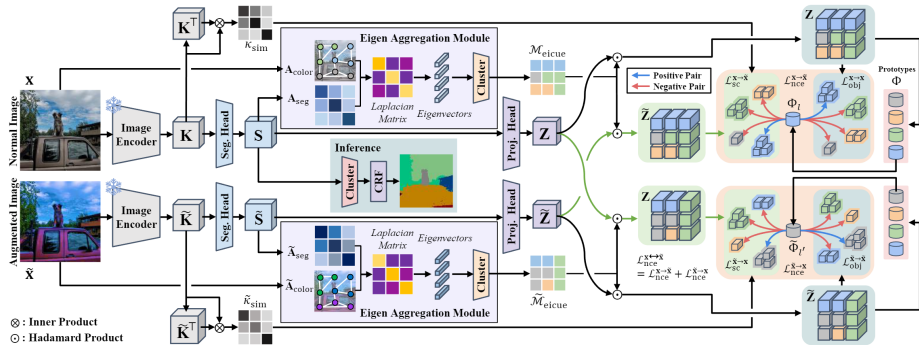


(a) Overview of *EAGLE*

Pipeline Components:

- ① **Backbone:** DINO ViT extracts features
- ② **EiCue Module:** Generates pseudo-masks via spectral clustering
- ③ **Projection Head:** Maps features to embedding space
- ④ **Training:**
 - *Linear Probe:* Predicts class labels (supervised by pseudo-masks)
 - *Cluster Probe:* Predicts cluster assignments (unsupervised)
 - *ObjNCELoss:* Refines feature space

EAGLE: Architecture



EAGLE: EiCue (Eigen Aggregation)

Improving Spectral Clustering Standard K-means on features is noisy. EAGLE uses **spectral clustering** on the Laplacian. **Affinity Construction:**

$$A = A_{color} + A_{seg} \quad (4)$$

- A_{color} : Local RGB differences (captures boundaries)
- $A_{seg} = SS^T$: Feature dot products (captures semantics)
- S : Learnable semantic features

Spectral Decomposition:

- Compute eigenvectors of Laplacian of A
- Run K-means on *eigenvectors*, not raw features
- **Result:** Coherent object masks

EAGLE: K-means vs. EiCue



Observation:

- **K-means:** Fragments objects based on texture/color
- **EiCue:** Groups semantically related regions (e.g., entire bed)

EAGLE: ObjNCELoss (Concept)

Object-Centric Contrastive Learning Standard InfoNCE works on *images* or *pixels*. EAGLE extends to *objects*.

- **Positive Pairs:** Pixel feature + prototype (centroid) of its object
- **Negative Pairs:** Pixel feature + prototypes of *other* objects

Goal

Pull pixels towards their object center, push away from other objects.

$$\mathcal{L}_{ObjNCE} = - \sum_{i \in \Omega} \log \frac{\exp(f_i \cdot c_{y_i} / \tau)}{\sum_{j=1}^K \exp(f_i \cdot c_j / \tau)} \quad (5)$$

Notation:

- f_i : Feature vector of pixel i
- c_{y_i} : Prototype vector of class y_i (assigned to pixel i)
- c_j : Prototype vector of class j
- K : Number of clusters/classes
- τ : Temperature scaling parameter

Effect: Refines feature space, making "thing" and "stuff" classes distinct.

Overview

- Curated subset of COCO-Stuff-164K
- 27 super-categories for semantic segmentation
- 15 "Thing" classes (countable objects)
- 12 "Stuff" classes (amorphous regions)

Thing Classes (Examples):

- Person, vehicle, animal
- Furniture, appliances
- Sports equipment

Stuff Classes (Examples):

- Sky, grass, water, road
- Wall, building, floor
- Tree, sand, snow

Binary Task

- **Foreground:** All "Thing" objects
- **Background:** All "Stuff" regions
- Primary benchmark for evaluating thing/stuff separation

Dataset Statistics:

- Complex real-world scenes
- Multiple objects per image

1. Mean Intersection over Union (mIoU)

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} = \frac{TP}{TP + FP + FN} \quad (6)$$

- Standard metric for segmentation accuracy
- Report mIoU for binary case (Foreground/Background)

2. Pixel Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

- Ratio of correctly classified pixels

EAGLE: Quantitative Results

Probe Type	mIoU	Accuracy
Linear Probe	41.19%	68.65%
Cluster Probe	24.40%	50.79%
Binary (Thing/Stuff)	62.03%	-

Table: COCO-Stuff-27 Performance

Observations:

- Binary mIoU (62.03%) significantly higher than TokenCut
- Linear Probe outperforms Cluster Probe
- EiCue effectively groups semantic parts

EAGLE: Qualitative Results



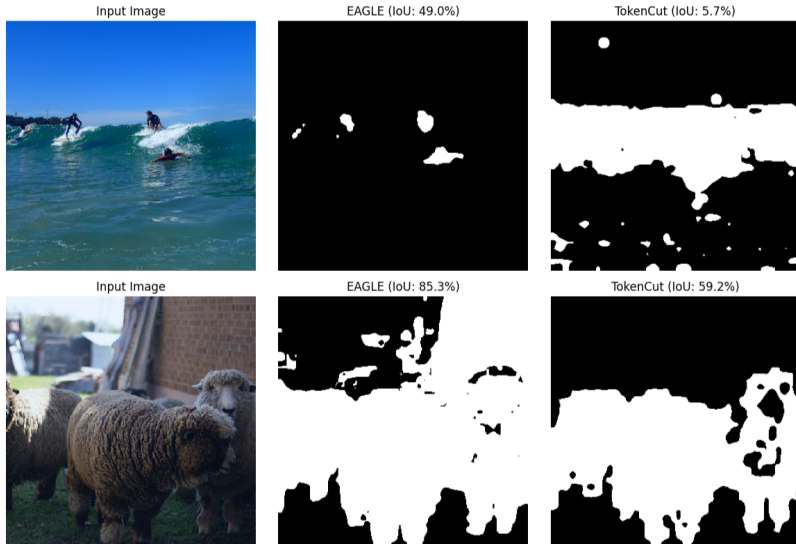
TokenCut vs Eagle: Differences

Aspect	TokenCut	EAGLE
Core Approach	Graph-based (Normalized Cuts)	Contrastive Learning (ObjNCELoss)
Training	Training-free (Inference only)	Self-Supervised (Requires training)
Optimization	Per-image spectral clustering	Learned weights (Feed-forward)
Inference Speed	Slow ($\sim 1.5s$) - Solve Eigenproblem	Fast ($\sim 0.13s$) - Linear projection
Segmentation	Binary (Foreground/Background)	Semantic (Multi-class capable)
Resolution	High (Patch-level graph)	Low (Feature-level maps)

Fundamental Difference

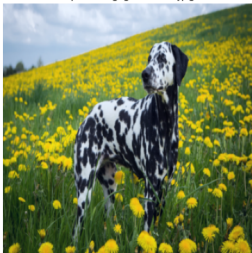
TokenCut treats segmentation as a **mathematical optimization problem** on a graph for each image individually. EAGLE treats it as a **representation learning problem**, learning to recognize objects across the entire dataset.

TokenCut vs Eagle: Qualitative Comparison (COCO Stuff)



TokenCut vs Eagle: Qualitative Comparison (Real Images)

Input: dog-grass (5).jpg



EAGLE Prediction



TokenCut Prediction



Input: football-player (6).jpg



EAGLE Prediction



TokenCut Prediction



Performance Comparison

Feature	TokenCut	EAGLE
Approach	Graph-based (N-Cut)	Contrastive Learning
Training	None	Required (SSL)
Binary mIoU	31.75%	62.03%
Resolution	High (Pixel-level)	Low (Feature-level)
Robustness	Low (Color-sensitive)	High (Semantic)

Verdict

EAGLE is superior for semantic segmentation accuracy (+30% mIoU).

Model	Time/Image	Bottleneck
TokenCut	~ 1.50 sec	Eigendecomposition ($O(N^3)$)
EAGLE	$\sim \mathbf{0.13}$ sec	Feed-forward pass ($O(N)$)

Scalability:

- **TokenCut:** $O(N^3)$ complexity - hard to scale to high-res
- **EAGLE:** $O(N)$ complexity - suitable for real-time

Verdict

EAGLE is suitable for real-time applications; TokenCut limited to offline processing.

Summary of Findings:

- ① EAGLE outperforms TokenCut by **+30% mIoU** on complex binary segmentation
- ② **Spectral Initialization (EiCue)** is critical for avoiding local optima
- ③ **Contrastive Learning (ObjNCELoss)** refines features for object-centric representations
- ④ **Training-free vs. Learned:** While TokenCut is attractive for zero-shot, EAGLE's learned representations provide robustness

1. Integration with Diffusion Models

- Leverage Stable Diffusion features (ODISE) for boundary precision
- Address EAGLE's low-resolution limitation

2. Multi-Scale Processing

- Combine TokenCut's edge sensitivity with EAGLE's semantic coherence
- Hybrid model for best of both worlds

3. Video Segmentation

- Extend EAGLE to video with temporal consistency loss
- Enforce consistent object IDs across frames

-  Wang, Y., et al. (2022). *Self-Supervised Transformers for Unsupervised Object Discovery using Normalized Cut*. CVPR 2022.
-  *Eigen Aggregation Learning for Unsupervised Semantic Segmentation*.
-  Caron, M., et al. (2021). *Emerging Properties in Self-Supervised Vision Transformers*. ICCV 2021.
-  Shi, J., & Malik, J. (2000). *Normalized Cuts and Image Segmentation*. IEEE TPAMI.
-  Caesar, H., et al. (2018). *COCO-Stuff: Thing and Stuff Classes in Context*. CVPR 2018.

Thank You